

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

Ярослав Войтович

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

Автор — Ярослав Войтович.

Ревізія 2026-08-26. Видання перше.

Ліцензія. Текст довідника — Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0). Приклади коду — MIT.

Довідник дозволено **вільно друкувати, копіювати і роздавати**, зокрема комерційним друком, за умови зазначення авторства; похідні матеріали поширюються на тих самих умовах. Повні тексти ліцензій — у репозиторії проекту.

Застереження. Матеріал описує стандартну інженерну роботу з мікроконтролерами: схемотехніку, код, протоколи, живлення, ремонт. Автор не несе відповідальності за наслідки застосування наведених відомостей. Робота з мережевим живленням, літійовими акумуляторами і радіопередавачами регулюється окремими нормами — дотримання їх лишається на відповідальності читача.

Технічні дані наведено за документацією Espressif Systems станом на дату ревізії. Кремній, документація і тулчейн змінюються — звіряйте критичні значення з першоджерелом.

Складено вільним програмним забезпеченням: Typst, pandoc. Гарнітури: Libertinus, DejaVu Sans Mono.

Зміст

59. Моніторинг датчиків з веб-інтерфейсом	1
60. Автономний логер	8
61. Радіоканал між двома платами	14
62. Керування виконавчими механізмами	18
63. Протокольний міст	24

59. Моніторинг датчиків з веб-інтерфейсом

Пристрій міряє температуру, вологість і тиск, тримає історію й показує її у браузері. Доступний за іменем, без пошуку IP-адреси.

Це базовий проект: на ньому зустрічаються Wi-Fi, I²C, веб-сервер, mDNS, зберігання стану й обробка помилок.

Постановка

Вхід: BME280 по I²C — температура, вологість, тиск.

Вихід: веб-сторінка з поточними значеннями і графіком за останні години; JSON для машинного зчитування.

Живлення: мережа через USB. Автономність не потрібна.

Поведінка при відмові: немає мережі — вимірювання продовжуються й накопичуються; датчик мовчить — пристрій живий і повідомляє про це.

Блок-схема

```
graph LR
    BME280["BME280 (0x76)"] -- "I2C" --> ESP32["ESP32 (кільцевий буфер на 720 записів)"]
    ESP32 -- "Wi-Fi" --> Router["роутер"]
    Router --> Browser["браузер (teplytsia.local)"]
```

Складові

Позиція	Кількість	Примітка
ESP32-S3-DevKitC-1 або classic DevKitC	1	будь-який із Wi-Fi
BME280, модуль I ² C	1	звірити адресу: 0x76 або 0x77
Резистори 4.7 кОм	2	підтягування I ² C, якщо немає на модулі
Дроти Dupont	4	
Корпус, живлення	—	розділ [54]

Ціни й наявність — у вкладиші ринку (P5).

Піни за платами

Проект розрахований на дві плати, і **піни в них різні**. Спочатку оберіть рядок, далі читайте схему з ним.

Сигнал	classic DevKitC	S3-DevKitC-1
SDA	GPI021	GPI08
SCL	GPI022	GPI09

НЕЗВОРОТНЕ

S3 GPI022 на S3 не існує. У S3 немає пінів 22–25 узагалі — це видно прямо в ESP-IDF, де маска дійсних пінів їх вирізає. Схема з GPI022, перенесена з classic, не запрацює й помилки не дасть: виклик конфігурації просто поверне ESP_ERR_INVALID_ARG, а шина лишиться німою.

Це загальне правило для всіх проектів цієї частини: **ВОМ із двома сімействами означає дві розпіновки**, і жодну з них не можна отримати з іншої заміною одного числа (додаток A).

Схема

Нижче — варіант для classic. Для S3 підставте піни з таблиці вище.

ESP32		BME280	
3V3	————	VCC	
GND	————	GND	
SDA	————	SDA	classic: GPIO21 S3: GPIO8
	└─[4.7к]─	3V3	
SCL	————	SCL	classic: GPIO22 S3: GPIO9
	└─[4.7к]─	3V3	

Підтягування обов'язкове (розділ [35]). Багато модулів BME280 мають власні резистори — тоді зовнішні не ставити.

Код

Проект ESP-IDF. Конфігурація через menuconfig або sdkconfig.defaults.

Читання датчика

Піни винесені в одне місце нагорі — так їх видно й так вони не розповзаються по коду:

```
// Піни за платою. Одне місце на весь проєкт.
#if CONFIG_IDF_TARGET_ESP32S3
# define PIN_SDA GPIO_NUM_8
# define PIN_SCL GPIO_NUM_9
#else // ESP32 classic
# define PIN_SDA GPIO_NUM_21
# define PIN_SCL GPIO_NUM_22
#endif
```

CONFIG_IDF_TARGET_* виставляє сама збірка після idf.py set-target, тож перемикання плати не потребує правок у коді (розділ [11]).

```
#include "driver/i2c_master.h"
#include "esp_log.h"

#define BME_ADDR 0x76
#define REG_ID 0xD0
#define REG_RESET 0xE0
#define REG_CTRL_H 0xF2
#define REG_CTRL_M 0xF4
#define REG_CONFIG 0xF5
#define REG_DATA 0xF7
#define REG_CALIB1 0x88
#define REG_CALIB2 0xE1

static const char *TAG = "BME";
static i2c_master_dev_handle_t bme;

// калібрувальні коефіцієнти живуть у самому датчику
static uint16_t T1; static int16_t T2, T3;
static uint16_t P1; static int16_t P2, P3, P4, P5, P6, P7, P8, P9;
static uint8_t H1, H3; static int16_t H2, H4, H5; static int8_t H6;
static int32_t t_fine;

static esp_err_t bme_read(uint8_t reg, uint8_t *buf, size_t len) {
    return i2c_master_transmit_receive(bme, &reg, 1, buf, len,
                                       pdMS_TO_TICKS(100));
}

static esp_err_t bme_write(uint8_t reg, uint8_t val) {
    uint8_t b[2] = { reg, val };
    return i2c_master_transmit(bme, b, 2, pdMS_TO_TICKS(100));
}

esp_err_t bme_init(i2c_master_bus_handle_t bus) {
    i2c_device_config_t cfg = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address = BME_ADDR,
        .scl_speed_hz = 100000,
    };
    ESP_RETURN_ON_ERROR(i2c_master_bus_add_device(bus, &cfg, &bme),
                        TAG, "не додано пристрій на шину");
}
```

```
// реєстр ідентифікації: доводить, що обмін працює (розділ 44)
uint8_t id = 0;
ESP_RETURN_ON_ERROR(bme_read(REG_ID, &id, 1), TAG, "немає відповіді");
if (id != 0x60) {
    ESP_LOGE(TAG, "чужий пристрій: id 0x%02x, очікували 0x60", id);
    return ESP_ERR_NOT_FOUND;
}

uint8_t c[26];
ESP_RETURN_ON_ERROR(bme_read(REG_CALIB1, c, 26), TAG, "калібрування");
T1 = c[0] | (c[1] << 8); T2 = c[2] | (c[3] << 8);
T3 = c[4] | (c[5] << 8); P1 = c[6] | (c[7] << 8);
P2 = c[8] | (c[9] << 8); P3 = c[10] | (c[11] << 8);
P4 = c[12] | (c[13] << 8); P5 = c[14] | (c[15] << 8);
P6 = c[16] | (c[17] << 8); P7 = c[18] | (c[19] << 8);
P8 = c[20] | (c[21] << 8); P9 = c[22] | (c[23] << 8);
H1 = c[25];

uint8_t h[7];
ESP_RETURN_ON_ERROR(bme_read(REG_CALIB2, h, 7), TAG, "калібрування H");
H2 = h[0] | (h[1] << 8);
H3 = h[2];
// старший байт H4 і H5 знаковий — розширення знака обов'язкове
H4 = ((int16_t)(int8_t)h[3] * 16) | (h[4] & 0x0F);
H5 = ((int16_t)(int8_t)h[5] * 16) | (h[4] >> 4);
H6 = (int8_t)h[6];

bme_write(REG_CTRL_H, 0x01); // osrs_h = x1
bme_write(REG_CONFIG, 0xA8); // t_sb 1000 мс, фільтр x4
bme_write(REG_CTRL_M, 0x27); // osrs_t x1, osrs_p x1, режим normal
ESP_LOGI(TAG, "BME280 знайдено і налаштовано");
return ESP_OK;
}
```

УВАГА

Реєстр ідентифікації читається **першим** і перевіряється. Це доводить, що обмін по шині працює, ще до того, як з'явиться перше значення (розділ [44]).

Без цієї перевірки несправна шина дає нулі, які виглядають як правдоподібні дані.

Порядок трьох записів не довільний: за datasheet зміна ctrl_hum набуває чинності **лише після запису в ctrl_meas**, тому ctrl_meas завжди останній. Записаний першим, він лишив би вологість невимірюною — і датчик віддавав би нулі, схожі на дані.

УВАГА

Два зсуви в розборі калібрування виглядають однаково і такими не є. H4 і H5 — **знакові** 16-бітові величини, і старший байт кожної береться зі знаком: (int16_t)(int8_t)h[3] * 16, а не h[3] << 4. Різниця виникає лише тоді, коли в старшому байті виставлено сьомий біт, тобто на частині екземплярів датчика — і виявляється як стабільно неправильна вологість при правильних температурі й тиску.

Це те місце, де варто звернутися з референсною реалізацією виробника, а не з чужим прикладом: у прикладах в інтернеті ця помилка трапляється частіше, ніж правильний варіант.

Перетворення сирих відліків — за формулами з datasheet. Без калібрувальних коефіцієнтів значення виглядають правдоподібно і є неправильними:

```
esp_err_t bme_measure(float *temp, float *hum, float *pres) {
    uint8_t d[8];
    ESP_RETURN_ON_ERROR(bme_read(REG_DATA, d, 8), TAG, "читання");

    int32_t adc_p = ((uint32_t)d[0] << 12) | ((uint32_t)d[1] << 4) | (d[2] >> 4);
    int32_t adc_t = ((uint32_t)d[3] << 12) | ((uint32_t)d[4] << 4) | (d[5] >> 4);
    int32_t adc_h = ((uint32_t)d[6] << 8) | (uint32_t)d[7];

    int32_t v1 = (((adc_t >> 3) - ((int32_t)T1 << 1))) * ((int32_t)T2) >> 11;
```

```

int32_t v2 = (((((adc_t >> 4) - ((int32_t)T1)) *
              ((adc_t >> 4) - ((int32_t)T1))) >> 12) * ((int32_t)T3)) >> 14;
t_fine = v1 + v2;
*temp = ((t_fine * 5 + 128) >> 8) / 100.0f;

int64_t p1 = ((int64_t)t_fine) - 128000;
int64_t p2 = p1 * p1 * (int64_t)P6 + ((p1 * (int64_t)P5) << 17)
            + (((int64_t)P4) << 35);
p1 = ((p1 * p1 * (int64_t)P3) >> 8) + ((p1 * (int64_t)P2) << 12);
p1 = (((((int64_t)1) << 47) + p1) * ((int64_t)P1)) >> 33;
if (p1 == 0) return ESP_ERR_INVALID_STATE;
int64_t p = 1048576 - adc_p;
p = (((p << 31) - p2) * 3125) / p1;
p1 = (((int64_t)P9) * (p >> 13) * (p >> 13)) >> 25;
p2 = (((int64_t)P8) * p) >> 19;
*pres = (((p + p1 + p2) >> 8) + (((int64_t)P7) << 4)) / 25600.0f;

int32_t h = t_fine - 76800;
h = (((((adc_h << 14) - (((int32_t)H4) << 20) - (((int32_t)H5) * h)) +
        16384) >> 15) * ((((((h * ((int32_t)H6)) >> 10) *
        ((h * ((int32_t)H3)) >> 11) + 32768)) >> 10) + 2097152) *
        ((int32_t)H2) + 8192) >> 14));
h -= (((((h >> 15) * (h >> 15)) >> 7) * ((int32_t)H1)) >> 4);
if (h < 0) h = 0;
if (h > 419430400) h = 419430400;
*hum = (h >> 12) / 1024.0f;
return ESP_OK;
}

```

Кільцевий буфер історії

```

#define ISTORIYA 720 // 12 годин при вимірюванні раз на хвилину

typedef struct {
    int64_t chas; // мікросекунди від старту
    float temp, hum, pres;
    bool valid;
} zapys_t;

static zapys_t istoriya[ISTORIYA];
static size_t idx = 0, kilkist = 0;
static SemaphoreHandle_t mutex;

static void dodaty(float t, float h, float p, bool ok) {
    xSemaphoreTake(mutex, portMAX_DELAY);
    istoriya[idx] = (zapys_t){ esp_timer_get_time(), t, h, p, ok };
    idx = (idx + 1) % ISTORIYA;
    if (kilkist < ISTORIYA) kilkist++;
    xSemaphoreGive(mutex);
}

```

Буфер виділяється **статично**, один раз. Ніякого malloc у циклі (розділ [30]).

Задача вимірювання

```
static void task_vymir(void *arg) {
    int pomylok_pospil = 0;
    while (1) {
        float t, h, p;
        esp_err_t err = bme_measure(&t, &h, &p);
        if (err == ESP_OK) {
            pomylok_pospil = 0;
            dodaty(t, h, p, true);
            ESP_LOGI(TAG, "%.2f °C, %.1f %, %.1f rПа", t, h, p);
        } else {
            pomylok_pospil++;
            dodaty(0, 0, 0, false);
            ESP_LOGW(TAG, "датчик не відповідає (%d посніль): %s",
                pomylok_pospil, esp_err_to_name(err));
            // деградуємо, а не перезавантажуємось (розділ 32)
        }
        ESP_LOGD(TAG, "вільно RAM: %lu, мінімум: %lu",
            esp_get_free_heap_size(), esp_get_minimum_free_heap_size());
        vTaskDelay(pdMS_TO_TICKS(60000));
    }
}
```

Датчик, що замовк, не зупиняє пристрій: записується позначка про збій, і робота триває. Веб-інтерфейс покаже, що дані застаріли.

Веб-сервер

```
static esp_err_t json_handler(httpd_req_t *req) {
    char *buf = malloc(16384);
    if (!buf) return httpd_resp_send_500(req);

    xSemaphoreTake(mutex, portMAX_DELAY);
    int n = snprintf(buf, 16384, "{\"zapysiv\":%u,\"dani\":[", kilkist);
    size_t start = (kilkist == ISTORIYA) ? idx : 0;
    bool pershyy = true; // ← не «i == 0», див. нижче
    for (size_t i = 0; i < kilkist && n < 16000; i++) {
        zapys_t *z = &istoriya[(start + i) % ISTORIYA];
        if (!z->valid) continue;
        n += snprintf(buf + n, 16384 - n,
            "%s{\"t\":%lld,\"temp\":%.2f,\"hum\":%.1f,\"pres\":%.1f}",
            pershyy ? "" : ",",
            z->chas / 1000000, z->temp, z->hum, z->pres);
        pershyy = false;
    }
    xSemaphoreGive(mutex);
    snprintf(buf + n, 16384 - n, "]}");

    httpd_resp_set_type(req, "application/json");
    esp_err_t r = httpd_resp_sendstr(req, buf);
    free(buf);
    return r;
}
```

УВАГА

Окремий прапорець `pershyy` замість перевірки `i == 0` — не педантизм. Записи зі збоєм пропускаються через `continue`, тож індекс циклу і номер **виведеного** елемента розходяться. Варіант `i ? ", " : ""` при першому ж збійному запису на початку історії поставить кому перед першим елементом, і JSON стане несинтаксичним: `"dani": [{...}]`.

Ламається це рівно тоді, коли датчик відмовив, — тобто саме тоді, коли на графік дивляться. Це типова форма помилки в цій книзі: код правильний для щасливого шляху й невірний для того, заради якого писався.

УВАГА

Буфер тут виділяється з купи й одразу звільняється — це **не** цикл виділень, а разова операція на запит. Різниця з правилом розділу [30] у тому, що частота низька й розмір фіксований.

Уважніше треба з іншим: обробник виконується в задачі веб-сервера з обмеженим стеком. Тому 16 КБ беруться з купи, а не оголошуються як локальний масив — інакше стек переповниться (розділ [30]).

Головна функція

```
void app_main(void) {
    ESP_LOGI(TAG, "старт, причина скидання: %d", esp_reset_reason());

    esp_err_t err = nvs_flash_init();
    if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        err = nvs_flash_init();
    }
    ESP_ERROR_CHECK(err);

    mutex = xSemaphoreCreateMutex();

    i2c_master_bus_config_t bus_cfg = {
        .i2c_port = I2C_NUM_0,
        .sda_io_num = PIN_SDA, // classic 21 / S3 8 — див. таблицю пінів
        .scl_io_num = PIN_SCL, // classic 22 / S3 9
        .clk_source = I2C_CLK_SRC_DEFAULT,
        .glitch_ignore_cnt = 7,
    };
    i2c_master_bus_handle_t bus;
    ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));

    if (bme_init(bus) != ESP_OK) {
        ESP_LOGE(TAG, "датчик не знайдено — працюємо без нього");
        // не ESP_ERROR_CHECK: веб-інтерфейс має піднятися й показати проблему
    }

    wifi_start(); // під'єднання з повторами, розділ 39
    mdns_init();
    mdns_hostname_set("teplytsia");
    mdns_service_add(NULL, "_http", "_tcp", 80, NULL, 0);

    web_start();
    xTaskCreate(task_vymir, "vymir", 4096, NULL, 5, NULL);
}
```

НЕЗВОРОТНЕ

ESP_ERROR_CHECK тут стоїть лише навколо NVS і створення шини — того, без чого пристрій не має сенсу.

Навколо ініціалізації датчика його **немає** свідомо: несправний датчик не повинен перетворювати пристрій на цеглинку. Веб-інтерфейс має піднятися й показати, що датчик мовчить (розділ [32]).

Збирання і перевірка

```
idf.py set-target esp32s3
idf.py menuconfig # Wi-Fi, розбивка флешу з OTA (розділ 18)
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

Порядок перевірки:

1. У лозі — BME280 знайдено і налаштовано. Немає — сканер I²C (розділ [35]).
2. Перше вимірювання з осмисленими значеннями.
3. teplytsia.local відкривається у браузері.
4. Від'єднати датчик на ходу: пристрій лишається живим, у лозі попередження, веб показує застарілі дані.
5. Вимкнути роутер: вимірювання тривають, після відновлення веб знову доступний.

6. Доба безперервної роботи: мінімум вільної пам'яті не зменшується (розділ [58]).

Розвиток

- **MQTT** замість або разом із веб-інтерфейсом (розділ [40]);
- **другий датчик** — DS18B20 на вулиці (розділ [37]);
- **OTA** — розбивку вже закладено (розділ [19]);
- **e-paper** для показу на місці (розділ [46]);
- **автономність**: перехід на deep sleep і ESP-NOW перетворює це на проєкт 60.

60. Автономний логер

Пристрій прокидається за розкладом, міряє, записує на картку й засинає. Живе від акумулятора місяцями. Головна тема проекту — **бюджет енергії** (розділ [06]) і надійність запису при зникненні живлення (розділ [49]).

Постановка

Вхід: BME280 (I²C) і DS18B20 (1-Wire) для виносного вимірювання.

Вихід: файл CSV на microSD, один рядок на вимірювання.

Живлення: 18650, ціль — не менше трьох місяців при вимірюванні раз на 15 хвилин.

Час: RTC DS3231 із власною батареєю — мережі немає, SNTP недоступний (розділ [40]).

Поведінка при відмові: картка недоступна — вимірювання накопичуються в RTC RAM; датчик мовчить — записується позначка; акумулятор розряджений — пристрій засинає назавжди, не псує картку.

Складові

Позиція	Кількість	Примітка
ESP32 classic або C3	1	не плата розробки для фінальної версії
BME280	1	I ² C
DS18B20 у зонді	1	1-Wire, резистор 4.7 кОм
DS3231 модуль RTC	1	I ² C, з батареєю CR2032
Модуль microSD	1	SPI
18650 з захистом	1	розділ [53]
TP4056 з захистом	1	розділ [53]
Перетворювач buck-boost на 3.3 В	1	ключове рішення, див. нижче
Резистори 4.7 кОм	3	I ² C і 1-Wire
MOSFET малий + резистори	1	ключ дільника вимірювання напруги

Піни за платами

Проекту потрібно вісім пінів, і на двох сімействах вони **різні повністю**, а не одним номером.

Сигнал	classic	C3
ADC дільника	GPIO34	GPIO3
Ключ дільника (вихід)	GPIO13	GPIO2 ⚠
I ² C SDA / SCL	GPIO21 / GPIO22	GPIO4 / GPIO5
1-Wire	GPIO4	GPIO1
SPI SCK / MOSI / MISO	GPIO18 / GPIO23 / GPIO19	GPIO6 / GPIO7 / GPIO10
SPI CS microSD	GPIO5	GPIO0

classic Четвірка 18/23/19/5 на classic — це рідні піни SPI3 (VSPI) один в один. **C3** На C3 рідними лишилися тільки SCK і MOSI: рідний MISO там GPIO2, а він уже пішов під ключ дільника. Для microSD це не коштує нічого — межа матриці 40 МГц (розділ [36]).

classic Один пін у цій таблиці варто назвати чесно: **GPIO5 на classic — теж strapping-пін**, як і GPIO2 на C3.

Позначки ⚠ він не отримав, і це послідовно: CS — вихід, у спокої високий, а strapping GPIO5 має внутрішнє

підтягування вгору й задає таймінги SDIO-веденого, яких у цьому проекті немає. Ризику тут немає, але правило книги — називати strapping-піни там, де їх беруть, — діє на обидві колонки.

НЕЗВОРОТНЕ

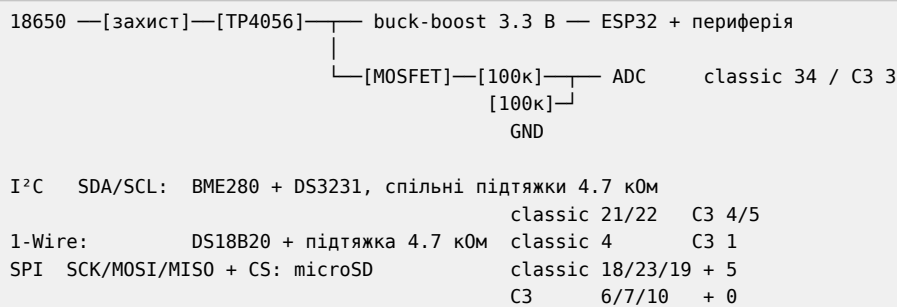
С3 Жодного з пінів GPI022, GPI023, GPI034 на С3 не існує. У С3 усього 22 піни, GPI00–GPI021, і класична розпіновка з розділу [07] переноситься на нього не заміною чисел, а перерозподілом усіх трьох шин. Ще гірше те, що з цих 22 пінів вільні далеко не всі: GPI012–GPI017 зайняті флешем, GPI018 і GPI019 — USB-Serial-JTAG, GPI020 і GPI021 — консоль UART0, а GPI02, GPI08, GPI09 — strapping. Лишається рівно вісім безумовно вільних: 0, 1, 3, 4, 5, 6, 7, 10.

Проекту треба **дев'ять**. Тому ключ дільника доводиться вішати на strapping-пін GPI02 — це припустимо лише тому, що він працює тут **виключно як вихід** і лише після старту (розділ [07]), а зовнішньої обв'язки на ньому немає.

Тобто **цей проект вичерпує С3 повністю й трохи більше**. Додати ще один датчик буде нікуди. Це саме той випадок, коли вибір чипа робиться на етапі схеми, а не після: на classic такої тісноти немає, і саме тому в BOM він стоїть першим.

Схема

Нижче — варіант для classic. Для С3 підставте піни з таблиці вище.



ЖИВЛЕННЯ

Buck-boost, а не LDO — головне рішення проекту. Звичайний LDO перестає давати 3.3 В, коли акумулятор просів до 3.8 В, і відсікає приблизно половину ємності. Buck-boost працює до 3.0 В.

Різниця — вдвічі за часом роботи, і вона більша за будь-яку оптимізацію коду (розділ [53]).

НЕЗВОРОТНЕ

Дільник вимірювання напруги вмикається транзистором. Постійно під'єднаний дільник із двох резисторів по 100 кОм бере на порядок більше, ніж чип уві сні (числа — розділ [33]), тобто стає головним споживачем усього пристрою.

Без ключа розрахунок на три місяці перетворюється на три тижні (розділ [53]).

Піни в коді

Уся розпіновка — в одному місці нагорі, а не розсіяна по викликах. Це не охайність заради охайності: саме розсіянні числа й роблять перенесення на інший чип неможливим без пропусків.

```
// Пини й канал ADC за платою (таблиця вище).
#ifdef CONFIG_IDF_TARGET_ESP32C3
# define PIN_SDA      GPIO_NUM_4
# define PIN_SCL      GPIO_NUM_5
# define PIN_I2C_WIRE GPIO_NUM_1
# define PIN_SCK      GPIO_NUM_6
# define PIN_MOSI     GPIO_NUM_7
# define PIN_MISO     GPIO_NUM_10
# define PIN_CS_SD     GPIO_NUM_0
# define PIN_DILNYK_EN GPIO_NUM_2 // ▲ strapping: лише вихід
# define ADC_CHANNEL   ADC_CHANNEL_3 // GPIO3
#else
// ESP32 classic
# define PIN_SDA      GPIO_NUM_21
# define PIN_SCL      GPIO_NUM_22
# define PIN_I2C_WIRE GPIO_NUM_4
# define PIN_SCK      GPIO_NUM_18
# define PIN_MOSI     GPIO_NUM_23
# define PIN_MISO     GPIO_NUM_19
# define PIN_CS_SD     GPIO_NUM_5
# define PIN_DILNYK_EN GPIO_NUM_13
# define ADC_CHANNEL   ADC_CHANNEL_6 // GPIO34 = ADC1_6
#endif
```

УВАГА

С3 Рядок PIN_DILNYK_EN на C3 — свідомий компроміс і єдине місце, де довелося взяти strapping-пін. Вільних більше немає, а GPIO2 тут працює **лише як вихід** і лише після старту, тож на завантаження не впливає (розділ [07]). Зовнішньої обв'язки на ньому бути не повинно — затвор MOSFET підключається напряму.

Якщо ця умова незручна, висновок той самий, що вище: беріть classic.

Стан, що переживає сон

Deep sleep — це перезавантаження: RAM втрачається, app_main починається спочатку (розділ [06]). Переживає сон лише RTC RAM.

```
RTC_DATA_ATTR static uint32_t nomer_cyklu = 0;
RTC_DATA_ATTR static uint32_t pomylok_karty = 0;
RTC_DATA_ATTR static zapys_t bufer[BUFER_ROZMIR]; // накопичення при збої
RTC_DATA_ATTR static uint8_t u_buferi = 0;
```

RTC RAM невелика — одиниці кілобайтів. Буфер на 20–30 записів у неї вміщається; більше — ні.

Головний цикл

```
void app_main(void) {
    nomer_cyklu++;
    ESP_LOGI(TAG, "цикл %lu, причина пробудження: %d",
              nomer_cyklu, esp_sleep_get_wakeup_cause());

    float napruga = zmiryaty_akumulyator(); // з ключем, див. нижче
    if (napruga < 3.2f) {
        ESP_LOGE(TAG, "акумулятор %.2f В — засинаємо назавжди", napruga);
        esp_deep_sleep_start(); // без таймера: не прокинеться
    }

    zapys_t z = { .chas = rtc_chas(), .napruga = napruga };
    z.ok_bme = (bme_measure(&z.temp, &z.hum, &z.pres) == ESP_OK);
    z.ok_ds = (ds18b20_read(&z.temp_zovni) == ESP_OK);

    if (!zapysaty_na_kartku(&z)) {
        pomylok_karty++;
        if (u_buferi < BUFER_ROZMIR) bufer[u_buferi++] = z;
        ESP_LOGW(TAG, "картка недоступна, у буфері %u", u_buferi);
    } else if (u_buferi > 0) {
        skynuty_bufer(); // дописати накопичене
    }
}
```

```
    zasnuty(15 * 60);
}
```

НЕЗВОРОТНЕ

Перевірка напруги стоїть **першою**, до будь-якої роботи з картою.

Причина: запис у карту при просілому живленні — найнадійніший спосіб пошкодити файлову систему FAT так, що втрачається не останній файл, а карта цілком (розділ [49]).

Розряджений акумулятор має призводити до чистого засинання, а не до спроби записати ще один рядок.

Вимірювання напруги через ключ

```
static float zmiryaty_akumulyator(void) {
    gpio_set_level(PIN_DILNYK_EN, 1);
    vTaskDelay(pdMS_TO_TICKS(10));           // дати зарядитися ємності

    int sum = 0;
    for (int i = 0; i < 32; i++) {           // усереднення (розділ 33)
        int raw;
        adc_oneshot_read(adc, ADC_CHANNEL, &raw);
        sum += raw;
    }
    gpio_set_level(PIN_DILNYK_EN, 0);       // вимкнути одразу

    int mv;
    adc_cal_raw_to_voltage(cali, sum / 32, &mv);
    return mv * 2.0f / 1000.0f;             // дільник 1:2
}
```

Надійний запис на карту

```
static bool zapysaty_na_kartku(const zapys_t *z) {
    if (sd_mount() != ESP_OK) return false;

    FILE *f = fopen(MOUNT "/dani.csv", "a");
    if (!f) { sd_unmount(); return false; }

    fprintf(f, "%lld,%.2f,%.1f,%.1f,%.2f,%.2f,%d,%d\n",
        z->chas, z->temp, z->hum, z->pres,
        z->temp_zovni, z->napruga, z->ok_bme, z->ok_ds);

    fflush(f);
    fsync(fileno(f));           // змусити записати на носій, а не в буфер
    fclose(f);
    sd_unmount();               // розмонтувати одразу
    return true;
}
```

НЕЗВОРОТНЕ

Чотири дії наприкінці — fflush, fsync, fclose, sd_unmount — виглядають надмірними і не є ними.

Файл, залишений відкритим, означає, що дані лежать у буфері в RAM. Deep sleep стирає RAM. Зникнення живлення — тим більше.

Правило логера: **відкрив, дописав рядок, закрив, розмонтував**. Ніколи не тримати файл відкритим між циклами (розділ [49]).

Засинання

```
static void zasnuty(uint32_t sekund) {
    // вимкнути все, що споживає
    sd_unmount();
    gpio_set_level(PIN_DILNYK_EN, 0);
    gpio_set_level(PIN_ZHYVLENNYA_PERYFERIYI, 0); // ключ живлення датчиків

    esp_sleep_enable_timer_wakeup((uint64_t)sekund * 1000000ULL);
    ESP_LOGI(TAG, "засинаємо на %lu c", sekund);
    esp_deep_sleep_start();
}
```

ЖИВЛЕННЯ

Живлення периферії вмикається ключем. Модуль microSD, BME280 і RTC споживають уві сні — разом це можуть бути мілі-, а не мікроампери.

Один MOSFET, що знімає живлення з усієї периферії на час сну, часто економить більше, ніж усі налаштування сну разом.

Виняток — RTC DS3231: він має власну батарею й лишається живим сам.

Бюджет енергії

Розрахунок для циклу раз на 15 хвилин:

Фаза	Час	Струм	Заряд
Пробудження й ініціалізація	300 мс	40 мА	12 мА·с
Вимірювання (DS18B20 — 750 мс)	900 мс	40 мА	36 мА·с
Запис на картку	400 мс	80 мА	32 мА·с
Сон	899 с	30 мкА	27 мА·с
Разом за цикл	900 с		≈107 мА·с

За добу: $96 \text{ циклів} \times 107 \text{ мА·с} \approx 10\,272 \text{ мА·с} \approx \mathbf{2.85 \text{ мА·год.}}$

З 18650 на 2500 мА·год, беручи 70 % на старіння і холод: $1750 / 2.85 \approx 614$ діб.

УВАГА

Розрахунок дає **понад рік**, і саме тому його треба перевірити вимірюванням (розділ [58]).

Практика зазвичай дає менше, і причини завжди ті самі: струм сну вищий за очікуваний (плата розробки!), холод з'їдає ємність, картка споживає більше при записі, ніж у datasheet.

Ціль «три місяці» в постановці — свідомо занижена. Розрахунок на рік із запасом утричі означає, що три місяці ви отримаєте навіть при неприємних сюрпризах.

Плата розробки для фінальної версії не годиться: USB-міст, стабілізатор і світлодіод споживають уві сні на порядки більше за чип. 20 мА замість 30 мкА перетворює рік на добу (розділ [06]).

Перевірка

1. Кілька циклів на столі з логом: значення осмислені, рядки в CSV з'являються.
2. **Виміряти струм сну** USB-тестером або, краще, шунтом і осцилографом (розділ [06]). Це головна перевірка проекту.
3. Вийняти картку на ходу — пристрій продовжує, накопичує в буфер.
4. Вставити назад — накопичене дописується.
5. Знизити напругу живлення лабораторним джерелом до 3.1 В — пристрій має заснути назавжди, не пошкодивши картку.
6. Вимикати живлення грубо, багато разів, зокрема під час запису (розділ [58]). Картка має лишатися читабельною.

7. Тиждень безперервної роботи з реальним акумулятором; порівняти реальне падіння напруги з розрахунком.

Розвиток

- **ESP-NOW** замість картки: передавати на приймач замість запису (розділ [42], проєкт 61) — прибирає картку й додає надійності;
- **e-rareg** для показу останнього значення без витрат енергії (розділ [46]);
- **сонячна панель** із контролером заряду;
- **ULP-співпроцесор** для пробудження за подією, а не за розкладом (розділ [03]).

61. Радіоканал між двома платами

Надійний обмін між двома пристроями без роутера, без інтернету й без інфраструктури. ESP-NOW із шифруванням, підтвердженням і контролем зв'язку.

Це основа для будь-якої системи «датчик — приймач» і найкорисніший проєкт для автономних вузлів (розділ [42]).

Постановка

Задача: передавач раз на хвилину надсилає вимірювання, приймач отримує, показує і віддає далі. Обидва мають знати, чи живий партнер.

Живлення: передавач від акумулятора (deep sleep між передачами), приймач від мережі.

Захист: шифрування; чужий пристрій не може ні прочитати, ні підробити пакет.

Поведінка при відмові: передавач не отримав підтвердження — повторює й накопичує; приймач не бачить пакетів — повідомляє про втрату вузла.

Чому ESP-NOW, а не Wi-Fi

Для передавача на батарейці різниця вирішальна (розділ [42]):

	Wi-Fi	ESP-NOW
Час до передачі	1–10 с	10 мс
Заряд на цикл	500 мА·с	5 мА·с
Потрібен роутер	так	ні

Два порядки економії — це різниця між місяцем і роками.

Формат пакета

Спільний заголовок для обох боків. Це те, що варто продумати один раз: змінити формат після розгортання означає перепрошити всі вузли.

```
#define PROTO_VERSIYA 1
#define TYP_VYMIR 1
#define TYP_PIDTVERDZH 2

typedef struct __attribute__((packed)) {
    uint8_t versiya; // версія протоколу — перше поле завжди
    uint8_t typ; // тип повідомлення
    uint8_t vuzol; // номер вузла
    uint8_t rezerv;
    uint32_t nomer; // лічильник пакетів: виявляє пропуски
    int32_t temp_x100; // ціле замість float — менше й переносніше
    uint16_t hum_x10;
    uint16_t napruga_mv;
} paket_t;

_Static_assert(sizeof(paket_t) <= 250, "ESP-NOW: максимум 250 байтів");
```

УВАГА

Три рішення в цій структурі, кожне з причиною.

versiya першим полем. Коли формат зміниться, старий приймач зможе розпізнати незнайому версію і не розібрати сміття як дані.

__attribute__((packed)) прибирає вирівнювання, яке компілятор додав би сам. Без нього структура на двох різних чипах може мати різний розмір.

Цілі замість `float`. Менший розмір, немає питань про представлення чисел, і на чипах без FPU дешевше (розділ [02]).

Передавач

```
static const uint8_t MAC_PRYIMACHA[6] = { 0x24, 0x6F, 0x28, 0x11, 0x22, 0x33 };

RTC_DATA_ATTR static uint32_t nomer = 0;
RTC_DATA_ATTR static paket_t bufer[10];
RTC_DATA_ATTR static uint8_t u_buferi = 0;

static volatile bool dostavleno = false;

static void on_sent(const esp_now_send_info_t *info,
                   esp_now_send_status_t status) {
    dostavleno = (status == ESP_NOW_SEND_SUCCESS);
}

static bool nadislaty(const paket_t *p) {
    dostavleno = false;
    if (esp_now_send(MAC_PRYIMACHA, (const uint8_t *)p, sizeof(*p)) != ESP_OK)
        return false;

    // чекати підтвердження рівня кадру — це не гарантія доставки,
    // але воно відрізняє «сусід почув» від «нікого немає»
    for (int i = 0; i < 50 && !dostavleno; i++)
        vTaskDelay(pdMS_TO_TICKS(2));
    return dostavleno;
}

void app_main(void) {
    radio_init(); // Wi-Fi у режимі STA, канал фіксований
    espnow_init_with_key();

    paket_t p = {
        .versiya = PROTO_VERSIYA,
        .typ = TYP_VYMIR,
        .vuzol = NOMER_VUZLA,
        .nomer = ++nomer,
    };
    zmiryaty(&p);

    if (!nadislaty(&p)) {
        if (u_buferi < 10) bufer[u_buferi++] = p;
        ESP_LOGW(TAG, "не доставлено, у буфері %u", u_buferi);
    } else {
        // дійшло — спробувати віддати накопичене
        while (u_buferi > 0 && nadislaty(&bufer[u_buferi - 1]))
            u_buferi--;
    }

    esp_sleep_enable_timer_wakeup(60ULL * 1000000);
    esp_deep_sleep_start();
}
```

УВАГА

`esp_now_send` повертається до фактичної передачі. Статус приходить у зворотний виклик `on_sent`, і саме його треба дочекатися перед засинанням — інакше чип засне посеред передачі.

Це та сама помилка, що з `uart_wait_tx_done` у RS-485 (розділ [34]), і проявляється так само: «інколи не доходить».

Сигнатура зворотного виклику змінилася: до ESP-IDF 5.4 першим аргументом був `const uint8_t *mac_addr`, тепер — `const esp_now_send_info_t *`. Приклади з інтернету старшого віку не зберуться, і повідомлення компілятора вкаже на тип, а не на причину.

Шифрування

```
static void espnow_init_with_key(void) {
    ESP_ERROR_CHECK(esp_now_init());
    ESP_ERROR_CHECK(esp_now_register_send_cb(on_sent));

    uint8_t pmk[16], lmk[16];
    nvs_read_key("pmk", pmk);          // з NVS, не з коду
    nvs_read_key("lmk", lmk);
    ESP_ERROR_CHECK(esp_now_set_pmk(pmk));

    esp_now_peer_info_t peer = { .channel = KANAL, .encrypt = true };
    memcpy(peer.peer_addr, MAC_PRYIMACHA, 6);
    memcpy(peer.lmk, lmk, 16);
    ESP_ERROR_CHECK(esp_now_add_peer(&peer));
}
```

НЕЗВОРОТНЕ

Ключі читаються з NVS, а не зашиті в код. Зашитий ключ дістається з дампа прошивки за п'ять хвилин (розділи [24], [50]).

І ключі мають бути **різні на кожен вузол**, а не один на всю систему: інакше захоплення одного датчика компрометує всю мережу.

Записуються при виробництві разом із номером вузла (розділ [21]).

Приймач

```
typedef struct {
    uint32_t ostanniy_nomer;
    int64_t ostanniy_chas;
    uint32_t vtracheno;
    bool zhyvyy;
} stan_vuzla_t;

static stan_vuzla_t vuzly[MAX_VUZLIV];
static QueueHandle_t cherga;

static void on_recv(const esp_now_recv_info_t *info,
                   const uint8_t *data, int len) {
    // виконується в контексті задачі Wi-Fi: скопіювати й вийти (розділ 42)
    if (len != sizeof(paket_t)) return;
    paket_t p;
    memcpy(&p, data, sizeof(p));
    xQueueSend(cherga, &p, 0);    // не FromISR: це задача, а не переривання
}

static void task_obrobka(void *arg) {
    paket_t p;
    while (xQueueReceive(cherga, &p, portMAX_DELAY) == pdTRUE) {
        if (p.versiya != PROTO_VERSIYA) {
            ESP_LOGW(TAG, "чужа версія протоколу: %u", p.versiya);
            continue;
        }
        if (p.vuzol >= MAX_VUZLIV) continue;

        stan_vuzla_t *v = &vuzly[p.vuzol];
        if (v->ostanniy_nomer && p.nomer > v->ostanniy_nomer + 1)
            v->vtracheno += p.nomer - v->ostanniy_nomer - 1;

        v->ostanniy_nomer = p.nomer;
        v->ostanniy_chas = esp_timer_get_time();
        v->zhyvyy = true;

        ESP_LOGI(TAG, "вузол %u: %.2f °C, %.1f %, %.2f B "
                    "(пакет %lu, втрачено %lu)",
                 p.vuzol, p.temp_x100 / 100.0f, p.hum_x10 / 10.0f,
                 p.napruga_mv / 1000.0f, p.nomer, v->vtracheno);
    }
}
```

```
vidaty_dali(&p);          // MQTT, веб, дисплей
    }
}
```

Контроль зв'язку

Приймач має сам помічати, що вузол зник, — не чекати, поки хтось питає:

```
static void task_kontrol(void *arg) {
    while (1) {
        int64_t teper = esp_timer_get_time();
        for (int i = 0; i < MAX_VUZLIV; i++) {
            stan_vuzla_t *v = &vuzly[i];
            if (!v->zhyvyy) continue;
            // три пропущені інтервали — вважаємо вузол мертвим
            if (teper - v->ostanniy_chas > 3 * 60 * 1000000LL) {
                v->zhyvyy = false;
                ESP_LOGE(TAG, "вузол %d не виходить на зв'язок", i);
                povidomyty_pro_vtratu(i);
            }
        }
        vTaskDelay(pdMS_TO_TICKS(10000));
    }
}
```

Лічильник `vtracheno` — безкоштовна оцінка якості зв'язку. Він одразу показує, чи проблема в конкретному вузлі, чи в загальних умовах.

Канал: головна складність розгортання

НЕЗВОРОТНЕ

Усі учасники мають бути **на одному каналі**. Якщо приймач також під'єднаний до Wi-Fi, його канал визначає **роутер** — і більшість роутерів обирають канал автоматично й змінюють його самі.

Це найчастіша причина «працювало на столі, на об'єкті перестало» (розділ [42]).

Три робочі варіанти:

Фіксований канал у роутері. Найпростіше, якщо роутер ваш.

Приймач без Wi-Fi. Тільки ESP-NOW на фіксованому каналі, дані далі йдуть по дроту.

Два чипи в приймачі. Один слухає ESP-NOW на фіксованому каналі, другий тримає Wi-Fi; між ними UART (розділ [34]). Найнадійніше і найдорожче.

Перевірка

1. Дві плати поруч: пакети доходять, лічильник росте без пропусків.
2. Рознести на робочу відстань, тиждень роботи: подивитися `vtracheno`.
3. Вимкнути передавач: приймач має за три хвилини повідомити про втрату.
4. Увімкнути назад: накопичене в буфері має дійти.
5. Спробувати надіслати пакет із третього пристрою без ключа: приймач має його не прийняти.
6. Виміряти струм передавача за цикл — має бути частки міліампер-секунди (розділ 60).

Розвиток

- **Кілька передавачів на один приймач** — структура вже готова (масив `vuzly`);
- **Двонаправлений обмін**: приймач надсилає команди у відповідь на пакет, поки передавач не заснув;
- **Заміна на LoRa** (розділ [43]), коли потрібні кілометри: формат пакета й логіка лишаються, змінюється транспорт;
- **Ретрансляція** через проміжний вузол для збільшення покриття.

62. Керування виконавчими механізмами

Пристрій вмикає й вимикає щось реальне: насос, клапан, нагрівач, двигун. Тема проєкту — **безпечна поведінка**, а не функціональність.

Різниця з попередніми проєктами принципова: помилка тут має фізичні наслідки. Насос, що не вимкнувся, заливає приміщення; нагрівач — підпалює.

Постановка

Задача: керувати насосом за розкладом і за датчиком рівня, з можливістю ручного втручання по мережі.

Виходи: реле насоса, індикація стану.

Входи: датчик рівня (поплашковий вимикач), кнопка «стоп», кнопка ручного пуску.

Безпека:

- насос вимкнений при зникненні живлення, при зависанні, при перезавантаженні;
- жорсткий ліміт часу безперервної роботи;
- захист від сухого ходу;
- аварійна зупинка апаратна, не програмна;
- втрата зв'язку не змінює стан механізму.

Складові й піни

Позиція	Кількість	Примітка
ESP32 classic DevKitC	1	схема нижче — для classic
Модуль реле з оптопарою	1	живлення котушки 5 В, розділ [47]
Аварійний вимикач	1	механічний, у силовому колі
Поплашковий вимикач	1	1. зовнішня підтяжка 10 кОм
Кнопки «пуск» і «стоп»	2	
Резистори 220 Ом, 10 кОм	по 1	

Сигнал	classic	S3	C3
Вихід на реле	GPIO25	GPIO4	GPIO4
Поплашковий вимикач	GPIO34	GPIO5	GPIO5
Кнопка «стоп»	GPIO26	GPIO6	GPIO6
Кнопка «пуск»	GPIO27	GPIO7	GPIO7

УВАГА

Схема й код нижче написані під classic, бо GPIO34 там **тільки вхідний** — і це принципово саме для цього проєкту: пін, який фізично не вміє бути виходом, не може випадково подати сигнал на реле через помилку в коді.

На S3 і C3 тільки-вхідних пінів немає взагалі (розділ [07]), тож цю властивість там не отримати. Замість неї доведеться покладатися на дисципліну коду — а в проєкті, де помилка заливає приміщення, це гірший захист.

Тому вибір classic тут не інерція, а рішення. Числа для інших сімейств наведені, щоб перенесення було свідомим, а не «замінити 34 на щось».

Безпечний стан: апаратно, до коду

НЕЗВОРОТНЕ

Питання, з якого починається проект: що станеться, якщо чип зникне просто зараз?

Відповідь має бути «насос вимкнеться», і забезпечується вона схемотехнікою, а не програмою (розділи [32], [47]).

Три речі, які треба зробити на платі:

Резистор 10 кОм, що утримує керувальний вхід у стані «вимкнено». Під час завантаження GPIO перебуває у високоімпедансному стані — без резистора вхід висить, і реле може ввімкнутися саме в момент старту. При boot loop (розділ [20]) це означає блимання насосом раз на секунду. Куди саме йде цей резистор — на землю чи на 3V3 — залежить від того, яким рівнем вмикається ваш модуль; таблиця в наступному підрозділі.

Реле з нормально розімкненими контактами. Знеструмлене реле = вимкнений насос. Ніколи навпаки.

Апаратний аварійний вимикач у розрив живлення насоса, не в логіку. Кнопка, що подає сигнал на GPIO, не працює, коли чип завис. Кнопка, що розриває коло, працює завжди.

Перевірка цих трьох речей — перший пункт випробувань, і робиться вона до написання логіки: подати живлення, поспостерігати за реле під час завантаження; висмикнути живлення під час роботи насоса.

Схема

Силове коло — усе послідовно, і аварійний вимикач у ньому фізично:

```
+12 В — [насос] — [реле: контакти NO] — [аварійний вимикач] — GND
```

Коло керування реле — окреме, від 5 В, а не від 3V3:

```
5 В — VCC модуля реле
GND — GND модуля (спільна з ESP32)

GPIO25 — [220 Ом] — IN модуля
      |
      [10 кОм] ← напрямок підтяжки залежить від модуля, див. нижче
      |
      GND або 3V3
```

Входи:

```
GPIO34 — поплавковий вимикач — GND (+ зовнішня підтяжка 10 кОм до 3V3!)
GPIO26 — кнопка «стоп» — GND (внутрішня підтяжка)
GPIO27 — кнопка «пуск» — GND
```

НЕЗВОРОТНЕ

Два питання до реленого модуля, які треба закрити до монтажу (розділ [47]).

Від чого живиться котушка. Більшість готових модулів розраховані на **5 В**. Від 3.3 В реле або не спрацює, або спрацюватиме через раз — класичне «іноді вмикається». Тому VCC модуля йде на 5 В, а не на 3V3.

Чи приймає вхід 3.3 В. Наявність оптопари цього **не гарантує**, хоч так часто пишуть. Світлодіод оптопар живиться від того самого VCC через резистор, підібраний під 5 В, і при керуванні від 3.3 В струм крізь нього падає — на частині модулів нижче порога спрацювання. Симптом той самий «іноді вмикається», лише причина інша.

Перевірити треба до монтажу, і це п'ять хвилин: подати 3.3 В на вхід і переконатися, що реле клацає надійно, а не на межі. Надійніше — зміряти струм через вхід і звірити з порогом оптопар в її datasheet. Якщо струму бракує — транзисторний ключ між GPIO і входом модуля (розділ [47]) або модуль, у якого вхідне коло розраховане на 3.3 В.

Яким рівнем вмикається. Багато модулів **інверсні**: реле вмикається логічним **нулем**. Від цього залежить напрямок підтяжки, і помилитися тут означає отримати рівно те, від чого весь цей розділ:

Модуль вмикається	Підтяжка 10 кОм	Стан під час завантаження
високим рівнем	на GND	реле вимкнене
низьким рівнем (інверсний)	на 3V3	реле вимкнене

Правило одне і формулюється не через напрямок, а через результат: **резистор утримує вхід у стані «вимкнено», поки GPIO ще не налаштований**. Перевіряється це не за схемою, а дослідом: подати живлення десять разів і подивитися на реле (пункт 1 випробувань нижче).

УВАГА

classic GPIO34 — тільки вхід і без вбудованого підтягування (розділ [07]). Поплавковому вимикачу потрібен зовнішній резистор 10 кОм до 3.3 В, інакше вхід бовтається й насос вмикається від наводок. Це саме той випадок, коли «датчик іноді спрацьовує сам» — не датчик, а відсутній резистор.

Автомат станів

Логіка керування будується як явний автомат, а не набором прапорців. Стан завжди один, переходи явні, і кожен перехід логується.

```
typedef enum {
    STAN_STOP,           // зупинено, чекає команди
    STAN_ROBOTA,         // насос працює
    STAN_AVARIYA,        // помилка, потрібне ручне скидання
    STAN_BLOKUVANNYA,    // пауза після роботи
} stan_t;

static stan_t stan = STAN_STOP;
static int64_t stan_vid;

static void perejty(stan_t novyy, const char *prychyna) {
    if (novyy == stan) return;
    ESP_LOGI(TAG, "%s -> %s: %s", nazva(stan), nazva(novyy), prychyna);
    stan = novyy;
    stan_vid = esp_timer_get_time();
    nasos_keruvaty(novyy == STAN_ROBOTA);
    onovyty_indykaciyu();
}
```

Логувати перехід із **причиною** — те, що робить лог придатним для розбору через місяць (розділ [25]).

Захисти

```
#define MAX_ROBOTY_S      600      // 10 хвилин безперервно
#define PAUZA_PISLYA_S   300      // 5 хвилин відпочинку
#define ZV_YAZOK_TAIMAUT 120      // втрата зв'язку

static void task_keruvannya(void *arg) {
    esp_task_wdt_add(NULL);
    while (1) {
        esp_task_wdt_reset();
        int64_t u_stani = (esp_timer_get_time() - stan_vid) / 1000000;

        // 1. Аварійна кнопка — найвищий пріоритет
        if (!gpio_get_level(PIN_STOP))
            perejty(STAN_AVARIYA, "натиснуто STOP");

        // 2. Сухий хід: працюємо, а рівня немає
        if (stan == STAN_ROBOTA && !riven_je() && u_stani > 10)
            perejty(STAN_AVARIYA, "сухий хід: немає рівня");

        // 3. Ліміт часу безперервної роботи
```



```

if (stan == STAN_ROBOTA && u_stani > MAX_ROBOTY_S)
    perejty(STAN_BLOKUVANNYA, "перевищено ліміт часу");

// 4. Кінець паузи
if (stan == STAN_BLOKUVANNYA && u_stani > PAUZA_PISLYA_S)
    perejty(STAN_STOP, "пауза завершена");

vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

НЕЗВОРОТНЕ

Ліміт часу безперервної роботи — найважливіший захист у проєкті.

Він рятує від помилок **керування**: збрехлого датчика, помилки в логіці, забутої ручної команди, зависання задачі. Насос, що працює десять хвилин замість двох, — незручність; насос, що працює добу, — аварія.

Ліміт ставиться завжди, навіть коли «за логікою такого не буває». Саме те, чого не буває за логікою, і трапляється.

НЕЗВОРОТНЕ

Чого програмний ліміт не рятує — і це треба знати точно.

Ліміт часу знімає сигнал із котушки. Далі все залежить від того, чи розмикається реле, коли котушка знеструмлена.

Відмова	Ліміт рятує?
датчик бреше, логіка помилилася, команда забута	так
задача зависла, а watchdog перезавантажив плату	так, якщо після скидання пін у безпечному стані
контакти реле зварилися	ні
пробитий ключ у силовому колі	ні
обрив у колі керування, реле лишилося замкненим	ні

Зварені контакти — не «залипання», яке можна відпустити. Це фізично з'єднаний метал: зняття струму з котушки не роз'єднує його, і пружина не долає зварювання. Реле в такому стані замкнене назавжди.

Саме тому в силовому колі цього проєкту стоїть **аварійний вимикач**, і саме тому він механічний і послідовний (схема вище). Він — єдиний захист від двох нижніх рядків таблиці, і жоден рядок коду його не замінює.

Якщо наслідок відмови серйозніший за калюжу — потрібне не програмне рішення, а схемне: друге реле в розрив, контактор із примусово зв'язаними контактами й контролем стану, або незалежне зняття живлення. Це вже царина норм на безпеку машин, і вибирати там за довідником загального призначення не можна.

НЕЗВОРОТНЕ

Стан AVARIYA не скидається сам. Вихід із нього — лише ручне втручання: кнопка або команда з мережі, і бажано після того, як людина подивилася, що сталося.

Автоматичне скидання аварії перетворює захист на затримку: пристрій циклічно вмикає насос, ловить сухий хід, чекає, вмикає знову.

Втрата зв'язку

НЕЗВОРОТНЕ

Втрата зв'язку не змінює стан механізму — це проєктне рішення, і воно має бути свідомим.

Два можливі варіанти, і обирати треба за наслідками:

Продовжити за локальною логікою. Правильно для насоса, що працює за розкладом і датчиком: мережа потрібна для нагляду, а не для роботи.

Зупинитися. Правильно для механізму, яким керує лише оператор: без зв'язку немає нагляду, тому рух припиняється.

Найгірший варіант — **не думати про це**: тоді поведінка визначається випадковим місцем у коді, де мережевий виклик заблокувався або ESP_ERROR_CHECK викликав перезавантаження (розділ [32]).

У цьому проєкті обрано перший варіант, і він записується в паспорт (розділ [56]).

Команди з мережі

```
static esp_err_t cmd_handler(httpd_req_t *req) {
    char buf[64];
    int n = httpd_req_recv(req, buf, sizeof(buf) - 1);
    if (n <= 0) return ESP_FAIL;
    buf[n] = 0;

    if (strcmp(buf, "pusk") == 0) {
        if (stan == STAN_AVARIYA)
            httpd_resp_sendstr(req, "avariya: potriben skydannya");
        else if (!riven_je())
            httpd_resp_sendstr(req, "nemaye rivnya");
        else {
            perejty(STAN_ROBOTA, "команда з мережі");
            httpd_resp_sendstr(req, "ok");
        }
    } else if (strcmp(buf, "stop") == 0) {
        perejty(STAN_STOP, "команда з мережі");
        httpd_resp_sendstr(req, "ok");
    } else if (strcmp(buf, "skydannya") == 0) {
        if (stan == STAN_AVARIYA) perejty(STAN_STOP, "ручне скидання");
        httpd_resp_sendstr(req, "ok");
    }
    return ESP_OK;
}
```

УВАГА

Команда **stop** виконується завжди й без умов. Це правило для будь-якого керування: зупинка не має перевірок, не має підтверджень і не залежить від поточного стану.

Усе інше може бути відхилене; зупинка — ніколи.

Обробник ідемпотентний: «стоп» двічі означає те саме, що один раз (розділ [40]).

Індикація

Стан має бути видимим на місці, без мережі:

Світлодіод	Стан
не горить	STOP
горить рівно	РОБОТА
блимає повільно	БЛОКУВАННЯ (пауза)
блимає швидко	АВАРІЯ

Людина біля пристрою мусить розуміти, що відбувається, не відкриваючи браузер (розділ [56]).

Перевірка

Порядок обов'язковий, і перші три пункти — до підключення насоса:

1. **Реле при завантаженні.** Подати живлення десять разів, спостерігати за реле. Жодного клацання під час старту.
2. **Зникнення живлення.** Висмикнути живлення при ввімкненому реле — реле має відпустити.
3. **Аварійний вимикач.** Розриває коло насоса незалежно від стану чипа.
4. **Ліміт часу:** запустити, дочекатися десяти хвилин — має перейти в блокування.
5. **Сухий хід:** запустити й від'єднати датчик рівня — аварія за десять секунд.
6. Аварія не скидається сама: почекати п'ять хвилин, стан лишається.
7. Втрата зв'язку: вимкнути роутер під час роботи — поведінка відповідає записаній у паспорті.
8. Зависання: викликати штучне зависання задачі — watchdog має перезавантажити чип, реле — відпустити.

НЕЗВОРОТНЕ

Пункти 1–3 виконуються **з від'єднаним насосом**. Перевіряти захисти на працюючому механізмі — це перевіряти їх наслідками.

Розвиток

- **Кілька механізмів** із взаємними блокуваннями;
- **Логування подій** у NVS: історія пусків і аварій переживає перезавантаження (розділ [18]);
- **Companion-схема** (розділ [57]): критичну логіку — на окремий контролер, ESP32 лишити зв'язок;
- **Датчик струму** (розділ [45]) для контролю, що насос справді крутиться, а не просто отримав живлення.

63. Протокольний міст

Пристрій, що з'єднує дві сторони, які не розмовляють одна з одною: обладнання з послідовним портом і мережу, RS-485 і Wi-Fi, CAN і MQTT.

Це найпоширеніша реальна роль ESP32 у чужій системі (розділ [57]), і проєкт зібраний як каркас, що налаштовується під конкретний випадок.

Постановка

Задача: прозора передавати дані між послідовним інтерфейсом і мережею, у обидва боки, з логуванням і можливістю налаштувати параметри без перепрошивки.

Варіанти, які покриває один каркас:

Міст	Один бік	Другий бік
Термінал по мережі	UART	TCP-сервер
Modbus TCP → RTU	RS-485	TCP-сервер
Телеметрія	UART або CAN	MQTT
Шлюз шини	CAN	Wi-Fi + MQTT

Поведінка при відмові: зникла мережа — дані з послідовного боку буферизуються; зник послідовний бік — мережевий клієнт отримує повідомлення, а не тишу.

Архітектура

Два незалежні напрямки, кожен зі своєю задачею і чергою. Це ключове рішення: жодна сторона не має блокувати іншу.

```
UART/RS-485/CAN          мережа
|                          |
[task_rx_serial] → cherga_do_merezhi → [task_tx_net]
[task_tx_serial] ← cherga_do_serial  ← [task_rx_net]
```

```
typedef struct {
    uint16_t dovzhyna;
    uint8_t dani[512];
} blok_t;

static QueueHandle_t do_merezhi; // від послідовного боку в мережу
static QueueHandle_t do_serial;  // з мережі в послідовний бік
```

УВАГА

Черги фіксованого розміру — це і буфер, і захист від перевантаження. Коли одна сторона швидша за іншу, черга заповнюється, і `xQueueSend` повертає помилку — тобто ви дізнаєтеся про перевантаження замість того, щоб мовчки з'їсти пам'ять.

Міст без обмеження буфера при відсутній мережі з'їдає всю купу за хвилини й падає (розділ [30]).

Послідовний бік

```
static void task_rx_serial(void *arg) {
    blok_t b;
    while (1) {
        int n = uart_read_bytes(UART_PORT, b.dani, sizeof(b.dani),
                                pdMS_TO_TICKS(20));

        if (n <= 0) continue;
        b.dovzhyna = n;
        if (xQueueSend(do_merezhi, &b, 0) != pdTRUE) {
            vtracheno_do_merezhi += n;
            ESP_LOGW(TAG, "черга в мережу повна, втрачено %d Б", n);
        }
    }
}
```

Читання з коротким таймаутом, а не порція фіксованого розміру: дані з послідовного порту приходять довільними шматками, і чекати повного буфера означає додавати затримку.

Розмір буфера драйвера беруть із запасом: на 115200 бод це близько 11 КБ на секунду, і 256 байтів переповнюються за 22 мілісекунди (розділ [34]).

RS-485: керування напрямком

```
static void nadislaty_serial(const blok_t *b) {
    #if REZHYM_RS485
        gpio_set_level(PIN_DE, 1);
    #endif
    uart_write_bytes(UART_PORT, b->dani, b->dovzhyna);
    #if REZHYM_RS485
        uart_wait_tx_done(UART_PORT, portMAX_DELAY);    // ОБОВ'ЯЗКОВО
        gpio_set_level(PIN_DE, 0);
    #endif
}
```

НЕЗВОРОТНЕ

uart_wait_tx_done перед перемиканням напрямку — найчастіша помилка роботи з RS-485 (розділ [34]).

uart_write_bytes лише кладе дані в буфер і повертається одразу. Перемикнути напрямок відразу після нього означає обрізати власну посилку посередині — і виглядає це як «інколи губляться відповіді», що збиває з пантелику надовго.

Мережевий бік: TCP-сервер

```
static void task_tcp(void *arg) {
    int listen_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_addr.s_addr = htonl(INADDR_ANY),
        .sin_port = htons(PORT),
    };
    bind(listen_sock, (struct sockaddr *)&addr, sizeof(addr));
    listen(listen_sock, 1);

    while (1) {
        struct sockaddr_in klient;
        socklen_t len = sizeof(klient);
        int sock = accept(listen_sock, (struct sockaddr *)&klient, &len);
        if (sock < 0) continue;

        ESP_LOGI(TAG, "клієнт під'єднався");
        // очистити чергу: клієнт не має отримати те, що накопичилося
        xQueueReset(do_merezhi);

        obsluhovuvaty(sock);    // до розриву з'єднання
        close(sock);
        ESP_LOGI(TAG, "клієнт від'єднався");
    }
}
```

```
}
}
```

УВАГА

xQueueReset при під'єднанні нового клієнта — важлива дрібниця. Без неї клієнт одразу отримує все, що накопичилося за час, поки ніхто не слухав: для термінала це сотні рядків старих даних.

Виняток — міст, де дані не можна втрачати. Тоді, навпаки, накопичене треба віддати, і черга має бути більшою.

Один клієнт одночасно — свідоме обмеження. Кілька клієнтів на одному послідовному порту означають перемішані відповіді: обладнання не знає, кому воно відповідає. Для Modbus це прямий шлях до хаосу.

Налаштування без перепрошивки

Міст, у якого швидкість зашита в код, доводиться перепрошивати при кожній зміні обладнання. Параметри йдуть у NVS (розділ [18]):

```
typedef struct {
    uint32_t baud;
    uint8_t data_bits, parity, stop_bits;
    uint8_t rezhym;           // UART, RS-485, CAN
    uint16_t tcp_port;
    char mqtt_uri[128];
    char mqtt_topic[64];
} nalashtuvannya_t;
```

Задаються через веб-форму (розділ [40]), зберігаються в NVS, читаються при старті. Скидання на заводські — довгим натисканням кнопки (розділ [39]).

Варіант CAN

```
static void task_rx_can(void *arg) {
    twai_message_t msg;
    while (1) {
        if (twai_receive(&msg, pdMS_TO_TICKS(100)) != ESP_OK) continue;

        blok_t b;
        b.dovzhyna = snprintf((char *)b.dani, sizeof(b.dani),
                               "%03lx:%d:", msg.identifier,
                               msg.data_length_code);
        for (int i = 0; i < msg.data_length_code; i++)
            b.dovzhyna += snprintf((char *)b.dani + b.dovzhyna,
                                   sizeof(b.dani) - b.dovzhyna,
                                   "%02x", msg.data[i]);
        b.dani[b.dovzhyna++] = '\n';
        xQueueSend(do_merezhi, &b, 0);
    }
}
```

Текстове представлення обрано свідомо: такий потік читається людиною в терміналі й розбирається будь-яким скриптом без домовленостей про двійковий формат.

НЕЗВОРОТНЕ

Міст на CAN за замовчуванням має працювати в режимі **LISTEN_ONLY** (розділ [38]).

Передача в чужу працюючу шину — дія з наслідками: пакет із чужим ідентифікатором може бути сприйнятий як команда керування. У техніці, що рухається або гріє, це не формальність.

Передача вмикається окремим свідомим налаштуванням, і за замовчуванням вона вимкнена.

Діагностика

Міст, який мовчки не працює, — найгірший варіант. Мінімум лічильників, доступних через веб:

```
static struct {
    uint32_t serial_rx, serial_tx;
    uint32_t net_rx, net_tx;
    uint32_t vtracheno_do_merezhi, vtracheno_do_serial;
    uint32_t pomylok_serial;
    int64_t ostannya_aktivnist;
} statystyka;
```

Ці сім чисел відповідають на головне питання діагностики: **на якому боці зупинилося**. Дані йдуть із послідовного, але не йдуть у мережу — проблема в мережі. Не йдуть із послідовного — проблема на тому боці або в налаштуваннях порту.

Без цих лічильників те саме з'ясується логічним аналізатором і годинами (розділ [28]).

Перевірка

1. **Заглушка:** з'єднати tx і rx між собою. Усе, що надіслано з мережі, має повернутися.
2. **Реальне обладнання:** підключити, звірити швидкість і формат.
3. **Обрив мережі** під час обміну: лічильник втрат росте, пристрій живий, після відновлення все працює.
4. **Обрив послідовного боку:** мережевий клієнт отримує повідомлення про це.
5. **Перевантаження:** подати потік швидший, ніж може прийняти другий бік. Черга має переповнитися з логом, а не з'їсти пам'ять.
6. **Доба роботи** з реальним трафіком: мінімум вільної пам'яті не зменшується (розділ [58]).

Розвиток

- **Modbus RTU ↔ TCP** зі штатним компонентом esp-modbus замість прозорого мосту (розділ [34]);
- **Кілька послідовних портів** одночасно;
- **Фільтрація** на боці CAN — приймати лише потрібні ідентифікатори, не турбуючи процесор рештою;
- **Буферизація у флеш** для мостів, де втрата даних неприйнятна;
- **Companion-роль** (розділ [57]): міст як постійна частина великої системи, з паспортом і версіонуванням (розділ [56]).

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

Ярослав Войтович

ревізія 2026-08-26

Знайшли помилку, застаріле значення або місце, де довідник вас підвів, — напишіть. Виправлення входять у наступну ревізію із зазначенням, що саме змінилося і чому.

github.com/ivoitovych/esp32-handbook-ua

CC BY-SA 4.0 · друкуйте і роздавайте вільно